

```
1 #include <windows.h>
2 #include <stdio.h> // for FILE I/O
3
4 #include <gl\glew.h> // for GLSL extensions IMPORTANT : This Line Should Be Before #include<gl\gl.h> And #include<gl\glu.h>
5
6 #include <gl/GL.h>
7
8 #include "vmath.h"
9
10 #pragma comment(lib,"glew32.lib")
11 #pragma comment(lib,"opengl32.lib")
12
13 #define WIN_WIDTH 800
14 #define WIN_HEIGHT 600
15
16 using namespace vmath;
17
18 enum
19 {
20     VDG_ATTRIBUTE_VERTEX = 0,
21     VDG_ATTRIBUTE_COLOR,
22     VDG_ATTRIBUTE_NORMAL,
23     VDG_ATTRIBUTE_TEXTURE0,
24 };
25
26 //Prototype Of WndProc() declared Globally
27 LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
28
29 //Global variable declarations
30 FILE *gpFile = NULL;
31
32 HWND ghwnd = NULL;
33 HDC ghdc = NULL;
34 HGLRC ghrc = NULL;
35
36 DWORD dwStyle;
37 WINDOWPLACEMENT wpPrev = { sizeof(WINDOWPLACEMENT) };
38
39 bool gbActiveWindow = false;
40 bool gbEscapeKeyIsPressed = false;
41 bool gbFullscreen = false;
42
43 GLuint gVertexShaderObject;
44 GLuint gFragmentShaderObject;
45 GLuint gShaderProgramObject;
46
47 //main()
48 int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int iCmdShow)
49 {
50     //function prototype
51     void initialize(void);
52     void uninitialize(void);
53     void display(void);
54 }
```

```
55     //variable declaration
56     WNDCLASSEX wndclass;
57     HWND hwnd;
58     MSG msg;
59     TCHAR szClassName[] = TEXT("OpenGLPP");
60     bool bDone = false;
61
62     //code
63     // create log file
64     if (fopen_s(&gpFile, "Log.txt", "w") != 0)
65     {
66         MessageBox(NULL, TEXT("Log File Can Not Be Created\nExiting ..."),
67             TEXT("Error"), MB_OK | MB_TOPMOST | MB_ICONSTOP);
68         exit(0);
69     }
70     else
71     {
72         fprintf(gpFile, "Log File Is Successfully Opened.\n");
73     }
74
75     //initializing members of struct WNDCLASSEX
76     wndclass.cbSize = sizeof(WNDCLASSEX);
77     wndclass.style = CS_HREDRAW | CS_VREDRAW | CS_OWNDC;
78     wndclass.cbClsExtra = 0;
79     wndclass.cbWndExtra = 0;
80     wndclass.hInstance = hInstance;
81     wndclass.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
82     wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
83     wndclass.hCursor = LoadCursor(NULL, IDC_ARROW);
84     wndclass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
85     wndclass.lpfnWndProc = WndProc;
86     wndclass.lpszClassName = szClassName;
87     wndclass.lpszMenuName = NULL;
88     //Registering Class
89     RegisterClassEx(&wndclass);
90
91     //Create Window
92     //Parallel to glutInitWindowSize(), glutInitWindowPosition() and
93     //glutCreateWindow() all three together
94     hwnd = CreateWindow(szClassName,
95         TEXT("OpenGL Programmable Pipeline Window"),
96         WS_OVERLAPPEDWINDOW,
97         100,
98         100,
99         WIN_WIDTH,
100        WIN_HEIGHT,
101        NULL,
102        NULL,
103        hInstance,
104        NULL);
105
106    ShowWindow(hwnd, iCmdShow);
107    SetForegroundWindow(hwnd);
108    SetFocus(hwnd);
```

```
109
110     //initialize
111     initialize();
112
113     //Message Loop
114     while (bDone == false) //Parallel to glutMainLoop();
115     {
116         if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
117         {
118             if (msg.message == WM_QUIT)
119                 bDone = true;
120             else
121             {
122                 TranslateMessage(&msg);
123                 DispatchMessage(&msg);
124             }
125         }
126         else
127         {
128             // rendering function
129             display();
130
131             if (gbActiveWindow == true)
132             {
133                 if (gbEscapeKeyIsPressed == true) //Continuation to
134                     glutLeaveMainLoop();
135                 bDone = true;
136             }
137         }
138     }
139     uninitialized();
140
141     return((int)msg.wParam);
142 }
143
144 //WndProc()
145 LRESULT CALLBACK WndProc(HWND hwnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
146 {
147     //function prototype
148     void resize(int, int);
149     void ToggleFullscreen(void);
150     void uninitialized(void);
151
152     //variable declarations
153     static WORD xMouse = NULL;
154     static WORD yMouse = NULL;
155
156     //code
157     switch (iMsg)
158     {
159     case WM_ACTIVATE:
160         if (HIWORD(wParam) == 0) //if 0, the window is active
161             gbActiveWindow = true;
162         else //if non-zero, the window is not active
163             gbActiveWindow = false;
```

```

164     break;
165     case WM_ERASEBKGD:
166         return(0);
167     case WM_SIZE: //Parallel to glutReshapeFunc();
168         resize(LOWORD(lParam), HIWORD(lParam)); //Parallel to glutReshapeFunc ↗
            (resize);
169     break;
170     case WM_KEYDOWN: //Parallel to glutKeyboardFunc();
171         switch (wParam)
172         {
173             case VK_ESCAPE: //case 27
174                 if (gbEscapeKeyPressed == false)
175                     gbEscapeKeyPressed = true; //Parallel to glutLeaveMainLoop ↗
                        ();
176                 break;
177             case 0x46: //for 'f' or 'F'
178                 if (gbFullscreen == false)
179                 {
180                     ToggleFullscreen();
181                     gbFullscreen = true;
182                 }
183                 else
184                 {
185                     ToggleFullscreen();
186                     gbFullscreen = false;
187                 }
188                 break;
189             default:
190                 break;
191         }
192     break;
193     case WM_LBUTTONDOWN: //Parallel to glutMouseFunc();
194     break;
195     case WM_CLOSE: //Parallel to glutCloseFunc();
196         uninitialize(); //Parallel to glutCloseFunc(uninitialize);
197     break;
198     case WM_DESTROY:
199         PostQuitMessage(0);
200     break;
201     default:
202     break;
203 }
204 return(DefWindowProc(hwnd, iMsg, wParam, lParam));
205 }
206
207 void ToggleFullscreen(void)
208 {
209     //variable declarations
210     MONITORINFO mi;
211
212     //code
213     if (gbFullscreen == false)
214     {
215         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
216         if (dwStyle & WS_OVERLAPPEDWINDOW)
217         {

```

```
218         mi = { sizeof(MONITORINFO) };
219         if (GetWindowPlacement(ghwnd, &wpPrev) && GetMonitorInfo
220             (MonitorFromWindow(ghwnd, MONITORINFOF_PRIMARY), &mi))
221         {
222             SetWindowLong(ghwnd, GWL_STYLE, dwStyle &
223                 ~WS_OVERLAPPEDWINDOW);
224             SetWindowPos(ghwnd, HWND_TOP, mi.rcMonitor.left,
225                 mi.rcMonitor.top, mi.rcMonitor.right - mi.rcMonitor.left,
226                 mi.rcMonitor.bottom - mi.rcMonitor.top, SWP_NOZORDER |
227                 SWP_FRAMECHANGED);
228         }
229     }
230     ShowCursor(FALSE);
231 }
232
233 else
234 {
235     //code
236     SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
237     SetWindowPlacement(ghwnd, &wpPrev);
238     SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
239         SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
240
241     ShowCursor(TRUE);
242 }
243 }
244
245 //FUNCTION DEFINITIONS
246 void initialize(void)
247 {
248     //function prototypes
249     void uninitialize(void);
250     void resize(int, int);
251
252     //variable declarations
253     PIXELFORMATDESCRIPTOR pfd;
254     int iPixelFormatIndex;
255
256     //code
257     ZeroMemory(&pfd, sizeof(PIXELFORMATDESCRIPTOR));
258
259     //Initialization of structure 'PIXELFORMATDESCRIPTOR'
260     //Parallel to glutInitDisplayMode()
261     pfd.nSize = sizeof(PIXELFORMATDESCRIPTOR);
262     pfd.nVersion = 1;
263     pfd.dwFlags = PFD_DRAW_TO_WINDOW | PFD_SUPPORT_OPENGL | PFD_DOUBLEBUFFER;
264     pfd.iPixelFormat = PFD_TYPE_RGBA;
265     pfd.cColorBits = 32;
266     pfd.cRedBits = 8;
267     pfd.cGreenBits = 8;
268     pfd.cBlueBits = 8;
269     pfd.cAlphaBits = 8;
270     pfd.cDepthBits = 32;
271
272     ghdc = GetDC(ghwnd);
```

```
268 //choose a pixel format which best matches with that of 'pfd'
269 iPixelFormatIndex = ChoosePixelFormat(ghdc, &pfd);
270 if (iPixelFormatIndex == 0)
271 {
272     ReleaseDC(ghwnd, ghdc);
273     ghdc = NULL;
274 }
275
276 //set the pixel format chosen above
277 if (SetPixelFormat(ghdc, iPixelFormatIndex, &pfd) == false)
278 {
279     ReleaseDC(ghwnd, ghdc);
280     ghdc = NULL;
281 }
282
283 //create OpenGL rendering context
284 ghrc = wglCreateContext(ghdc);
285 if (ghrc == NULL)
286 {
287     ReleaseDC(ghwnd, ghdc);
288     ghdc = NULL;
289 }
290
291 //make the rendering context created above as current n the current hdc
292 if (wglMakeCurrent(ghdc, ghrc) == false)
293 {
294     wglDeleteContext(ghrc);
295     ghrc = NULL;
296     ReleaseDC(ghwnd, ghdc);
297     ghdc = NULL;
298 }
299
300 // GLEW Initialization Code For GLSL ( IMPORTANT : It Must Be Here. Means ↗
    After Creating OpenGL Context But Before Using Any OpenGL Function )
301 GLenum glew_error = glewInit();
302 if (glew_error != GLEW_OK)
303 {
304     wglDeleteContext(ghrc);
305     ghrc = NULL;
306     ReleaseDC(ghwnd, ghdc);
307     ghdc = NULL;
308 }
309
310 // *** VERTEX SHADER ***
311 // create shader
312 glVertexShaderObject = glCreateShader(GL_VERTEX_SHADER);
313
314 // provide source code to shader
315 const GLchar *vertexShaderSourceCode =
316     "void main(void)" \
317     "#version 450" \
318     "\n" \
319     "{" \
320     "}";
321 glShaderSource(glVertexShaderObject, 1, (const GLchar **)
    &vertexShaderSourceCode, NULL);
```

```
322
323     // compile shader
324     glCompileShader(gVertexShaderObject);
325
326     // *** FRAGMENT SHADER ***
327     // create shader
328     gFragmentShaderObject = glCreateShader(GL_FRAGMENT_SHADER);
329
330     // provide source code to shader
331     const GLchar *fragmentShaderSourceCode =
332         "void main(void)" \
333         "#version 450" \
334         "\n" \
335         "{" \
336         "}";
337     glShaderSource(gFragmentShaderObject, 1, (const GLchar **)
338         &fragmentShaderSourceCode, NULL);
339
340     // compile shader
341     glCompileShader(gFragmentShaderObject);
342
343     // *** SHADER PROGRAM ***
344     // create
345     gShaderProgramObject = glCreateProgram();
346
347     // attach vertex shader to shader program
348     glAttachShader(gShaderProgramObject, gVertexShaderObject);
349
350     // attach fragment shader to shader program
351     glAttachShader(gShaderProgramObject, gFragmentShaderObject);
352
353     // link shader
354     glLinkProgram(gShaderProgramObject);
355
356     glShadeModel(GL_SMOOTH);
357     // set-up depth buffer
358     glClearDepth(1.0f);
359     // enable depth testing
360     glEnable(GL_DEPTH_TEST);
361     // depth test to do
362     glDepthFunc(GL_LEQUAL);
363     // set really nice perspective calculations ?
364     glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST);
365     // We will always cull back faces for better performance
366     glEnable(GL_CULL_FACE);
367
368     // set background color to which it will display even if it will empty.
369     THIS LINE CAN BE IN drawRect().
370     glClearColor(0.0f, 0.0f, 1.0f, 0.0f); // blue
371
372     // resize
373     resize(WIN_WIDTH, WIN_HEIGHT);
374 }
375
376 void display(void)
377 {
```

```
376     //code
377     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
378
379     // start using OpenGL program object
380     glUseProgram(gShaderProgramObject);
381
382     // OpenGL Drawing
383
384     // stop using OpenGL program object
385     glUseProgram(0);
386
387     SwapBuffers(ghdc);
388 }
389
390 void resize(int width, int height)
391 {
392     //code
393     if (height == 0)
394         height = 1;
395     glViewport(0, 0, (GLsizei)width, (GLsizei)height);
396 }
397
398 void uninitialize(void)
399 {
400     //UNINITIALIZATION CODE
401     if (gbFullscreen == true)
402     {
403         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
404         SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
405         SetWindowPlacement(ghwnd, &wpPrev);
406         SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
407             SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
408
409         ShowCursor(TRUE);
410     }
411
412     // detach vertex shader from shader program object
413     glDetachShader(gShaderProgramObject, gVertexShaderObject);
414     // detach fragment shader from shader program object
415     glDetachShader(gShaderProgramObject, gFragmentShaderObject);
416
417     // delete vertex shader object
418     glDeleteShader(gVertexShaderObject);
419     gVertexShaderObject = 0;
420     // delete fragment shader object
421     glDeleteShader(gFragmentShaderObject);
422     gFragmentShaderObject = 0;
423
424     // delete shader program object
425     glDeleteProgram(gShaderProgramObject);
426     gShaderProgramObject = 0;
427
428     // unlink shader program
429     glUseProgram(0);
430 }
```

```
431     //Deselect the rendering context
432     wglMakeCurrent(NULL, NULL);
433
434     //Delete the rendering context
435     wglDeleteContext(ghrc);
436     ghrc = NULL;
437
438     //Delete the device context
439     ReleaseDC(ghwnd, ghdc);
440     ghdc = NULL;
441
442     if (gpFile)
443     {
444         fprintf(gpFile, "Log File Is Successfully Closed.\n");
445         fclose(gpFile);
446         gpFile = NULL;
447     }
448 }
449
```